

Dismantling GayFemBoy: Reverse Engineering a 2025 Mirai IoT Botnet Derivative

Alexander Schoolcraft
adscho3199@ung.edu
College of Math and Sciences
University of North Georgia
Dahlonega, Georgia, USA

```
C2 DOMAIN 2: "i-kiss-boys.com" - second hardcoded C2 fallback...
s_i-kiss-boys.com_0806eda4 XREF[1]: scanner_build_target_list_from_c...
0806eda4 69 2d 6b ds "i-kiss-boys.com"
69 73 73
2d 62 6f ...

C2 DOMAIN 3: "furry-femboys.top" - third hardcoded C2 fallback...
s_furry-femboys.top_0806edb4 XREF[1]: scanner_build_target_list_from_c...
0806edb4 66 75 72 ds "furry-femboys.top"
72 79 2d
66 65 6d ...

C2 DOMAIN 4: "twinkfinder.nl" - fourth hardcoded C2 domain. T...
s_twinkfinder.nl_0806edc6 XREF[1]: scanner_build_target_list_from_c...
0806edc6 74 77 69 ds "twinkfinder.nl"
6e 6b 66
69 6e 64 ...
```

Figure 1. Some of the target URLs used in the final stage

Abstract

This work presents a reverse engineering analysis of a 2024 Mirai-derived IoT botnet sample, referred to as “GayFemBoy.” The malware exhibits a multi-stage execution pipeline involving an initial loader, in-memory unpacking of a second-stage payload, and a final stage responsible for command-and-control (C2) communication and target enumeration. Using a combination of static analysis in Ghidra and dynamic analysis via GDB, strace, and network instrumentation, the execution flow, syscall behavior, and runtime memory transformations were reconstructed. Key findings include self-deletion behavior, dynamic remapping of executable regions, staged payload loading, and hardcoded fallback C2 infrastructure. This analysis demonstrates the continued evolution of Mirai-based malware and highlights the importance of combined static and dynamic techniques for analyzing multi-stage IoT threats.

1 Background

Mirai is a well-known IoT botnet malware family [1] that targets embedded Linux devices by exploiting weak authentication and exposed services. Since its public release, numerous variants and derivatives have emerged, often incorporating obfuscation, staging mechanisms, and updated command-and-control (C2) infrastructure.

Modern Mirai variants frequently employ multi-stage payload delivery to evade static detection and complicate analysis. These stages may include a lightweight loader, a secondary unpacking or decryption phase, and a final payload responsible for network propagation or attack execution.

The sample analyzed in this work represents a recent Mirai derivative exhibiting staged execution, runtime memory manipulation, and network-based command retrieval. Its behavior aligns with contemporary trends in IoT malware, including ephemeral execution artifacts, dynamic payload loading, and fallback C2 mechanisms.

2 Methodology

The analysis was conducted using a combination of static and dynamic reverse engineering techniques within an isolated virtualized environment.

Static analysis was performed using Ghidra to identify program structure, function boundaries, and key syscall wrappers. Particular attention was given to functions associated with network communication, including socket creation, connection handling, and data reception.

Dynamic analysis was conducted using GDB with breakpoints placed on critical syscall interfaces, including `int 0x80` transitions and socket-related operations. Execution

Following process creation, the malware emits a short message to standard output, as shown in Figure 4. Analysis of the corresponding code path indicates that this output is written immediately after a `fork()` call and serves as a liveness signal from the child process to the parent. This behavior confirms successful daemonization and provides a synchronization point during execution.

Overall, the sample demonstrates a layered architecture combining obfuscation, staged payload delivery, and resilient C2 mechanisms. Together, these behaviors illustrate a coordinated design focused on stealth, persistence, and exclusive host control, consistent with modern Mirai-derived botnet architectures.

4 Conclusion

This work demonstrates the effectiveness of combining static and dynamic analysis techniques when reversing modern

multi-stage malware. The analyzed Mirai-derived sample employs runtime memory manipulation, staged execution, and resilient network communication strategies to evade analysis and detection.

By reconstructing the execution flow across multiple stages, it was possible to identify key behavioral components, including payload loading, syscall abstraction, and C2 infrastructure. These findings highlight the continued evolution of IoT botnets and the increasing complexity of their implementation.

Future work includes deeper analysis of the final-stage payload, automated extraction of staged binaries, and integration of these techniques into scalable malware analysis frameworks.

References

- [1] ANTONAKAKIS, M., ET AL. Mirai source code. <https://github.com/jgamblin/Mirai-Source-Code>, 2016.